National College of Ireland

Project Submission Sheet

Student Name: Meenakshi

Student ID: X24235415

Programme: Msc Cloud Computing **Year:** 2025

Module: Cloud DevSecOps

Lecturer: Vikas Sahni

Submission Due Date: 08/12/2025

Project Title: Parking Management System with Automated Security-Centric CI/CD Pipeline

Word Count: ……

I hereby certify that the information contained in this (my submission) is information pertaining to research I conducted for this project. All information other than my own contribution will be fully referenced and listed in the relevant bibliography section at the rear of the project.
ALL internet material must be referenced in the references section. Students are encouraged to use the Harvard Referencing Standard supplied by the Library. To use other author's written or electronic work is illegal (plagiarism) and may result in disciplinary action. Students may be required to undergo a viva (oral examination) if there is suspicion about the validity of their submitted work.

Signature: ……

Date: ……

PLEASE READ THE FOLLOWING INSTRUCTIONS:

1. Please attach a completed copy of this sheet to each project (including multiple copies).
2. Projects should be submitted to your Programme Coordinator.
3. **You must ensure that you retain a HARD COPY of ALL projects**, both for your own reference and in case a project is lost or mislaid. It is not sufficient to keep a copy on computer. Please do not bind projects or place in covers unless specifically requested.
4. You must ensure that all projects are submitted to your Programme Coordinator on or before the required submission date. **Late submissions will incur penalties.**
5. All projects must be submitted and passed in order to successfully complete the year. **Any project/assignment not submitted will be marked as a fail.**
6. **Please check that you read AI and Academic Integrity Acknowledgement Supplments in this document**

Office Use Only	
Signature:	Meenakshi
Date:	08/12/2025
Penalty Applied (if applicable):	

AI Acknowledgement Supplement

[Insert Module Name]

[Insert Title of your assignment]

Your Name/Student Number	Course	Date
Meenakshi	Msc Cloud Computing	08/12/2025

This section is a supplement to the main assignment, to be used if AI was used in any capacity in the creation of your assignment; if you have queries about how to do this, please contact your lecturer. For an example of how to fill these sections out, please click here.

AI Acknowledgment

This section acknowledges the AI tools that were utilized in the process of completing this assignment.

Tool Name	Brief Description	Link to tool

Description of AI Usage

This section provides a more detailed description of how the AI tools were used in the assignment. It includes information about the prompts given to the AI tool, the responses received, and how these responses were utilized or modified in the assignment. **One table should be used for each tool used**.

[Insert Tool Name]	
[Insert Description of use]	
[Insert Sample prompt]	[Insert Sample response]

Evidence of AI Usage

This section includes evidence of significant prompts and responses used or generated through the AI tool. It should provide a clear understanding of the extent to which the AI tool was used in the assignment. Evidence may be attached via screenshots or text.

Additional Evidence:

[Place evidence here]

Additional Evidence:

[Place evidence here]

Parking Management System with Automated Security-Centric CI/CD Pipeline

Meenakshi
Student ID: x24235415
MSc in Cloud Computing
National College of Ireland, Dublin
Email: x24235415@student.ncirl.ie

***Abstract*—This research paper presents the design, development, and deployment of a cloud-hosted Parking Management System that demonstrates modern DevSecOps practices. The application was developed on the Django 4.2.9 framework and deployed on AWS Elastic Beanstalk, using Gunicorn as the WSGI server and Nginx for reverse proxy functionality. A four-stage comprehensive CI/CD pipeline was implemented using GitHub Actions, with automated security scanning through SonarCloud for static analysis and OWASP ZAP for dynamic application security testing. The system efficiently handles parking operations, including user registration, tracking of vehicles, real-time parking slot management, and automatic calculations of bills. Security validation confirmed zero critical vulnerabilities, while the identified medium and low severity issues were documented for future remediation. This successfully proved the integration of security throughout the software development lifecycle without sacrificing deployment efficiency or code quality standards. Keywords: Parking Management, Django, AWS Elastic Beanstalk, DevSecOps, CI/CD, OWASP ZAP, SonarCloud, Cloud Security**

I. Introduction

A. Project Motivation

Traditional parking management systems are wrought with inefficiencies, including manual processes, timing calculations that are often inaccurate, poor space utilization, and a lack of real-time tracking. Most parking facilities still depend on paper or basic spreadsheet tracking, which brings about revenue leakage, disgruntled customers, and inefficiency in operations. The absence of automated systems results in inaccurate billing, difficulty in tracking peak usage periods, and limited capacity for data-driven decision making. The increasing adoption of cloud technologies presents an opportunity to transform parking management through automated, scalable, and secure web applications. However, many cloud deployments focus mainly on functionality at the expense of security, hence putting systems into potential vulnerabilities. This project addresses both operational efficiency and security concerns by implementing a cloud-based solution with integrated security validation throughout the development lifecycle.

B. Project Objectives

This project had the following main objectives: 1. Application Development: Design a web-based application using the Django framework, which shall provide full CRUD operations for Parking Management, including user authentication, vehicle registration, tracking of parking sessions, and automatic calculations of bills. 2. Security Implementation: Apply OWASP Top 10 security principles by utilizing Django's built-in security features, which provide protection against SQL injection, XSS, CSRF security, secure authentication mechanisms, etc. 3. CI/CD Pipeline: Design a strong four-stage GitHub Actions pipeline that will include: '*' Static code analysis using SonarCloud - Stage 1 * Stage 2: Dynamic security testing with OWASP ZAP 3*. Stage: Automated testing coverage reporting * Stage 4: Automated deployment to AWS Elastic Beanstalk 4. Cloud Deployment: The application will be deployed on AWS Elastic Beanstalk using SQLite database. This involves auto-scaling, load balancing, and proper environment configuration for production readiness. 5. Extensive Testing: Perform extensive testing for test coverage, emphasizing the most critical business logic such as parking duration calculations, accurate billing, user authentication, and access control validation.

C. Application Overview

The Parking Management System is a centralized platform where parking operations can be managed by three different types of users: Administrators: Full system access including user management, parking slot configuration, revenue reporting, and system configuration * Parking Attendants: For operational access, vehicle registration, parking session management, payment processing, and real-time occupancy monitoring * Regular Users: self-service portal for tracking vehicle registration, parking session history, and current parking status Core Functionalities: Real-time tracking of parking slot availability * Automated timing and billing calculation Vehicle registration and management system * Implementation of role-based access control * Complete parking history and reporting * Email notification system for parking reminders Security Framework: * Django authentication with PBKDF2 password hashing * Prevention of SQL injection through Django ORM * XSS protection via template auto-escaping * CSRF protection on all state-changing operations * Secure file upload handling with validation • Environment-based configuration management * Complete logging for security auditing

II. Continuous Integration, Continuous Delivery and Deployment

parking-application.eba-grmnmnvr.us-east-1.elasticbeanstalk.com

A. Pipeline Architecture

The CI/CD pipeline on GitHub Actions automates the process of checking, testing, and deployment whenever there has been a change in the project. This happens in two situations: Whenever new code is pushed to the main branch and Whenever a pull request is created or updated. This ensures that all code, regardless of its origin, undergoes the same quality checks before it can be merged or deployed. The pipeline is organized into a series of stages, such as setup, testing, security scanning, and deployment. Each of these stages runs in a fail-fast manner: until one stage is complete, the next stage will not start. If any of these stages fails, the pipeline is immediately stopped to prevent faulty, insecure, or poorly written code from moving further down the deployment path.

By stopping early and providing fast feedback, the pipeline helps developers identify and fix issues much sooner. At the same time, it keeps the main branch stable and reduces the risk of deploying broken or unsafe code in production.

B. Stage 1: SonarCloud Analysis

Static code analysis by SonarCloud constitutes the first stage of the pipeline. This step assesses the codebase for maintainability issues, potential security risks, and technical debt. SonarCloud looks at code complexity, duplicated logic, naming conventions, and also vulnerabilities such as SQL injections, cross-site scripting, weak cryptography, and exposed secrets. Each of the detected issues are graded in severity, with the Project's Quality Gate blocking any commit that introduces high-risk or critical problems. Linked with GitHub, developers are able to see issues right within pull requests and track code quality trends over time.

C. Stage 2: Security and Quality Checks

This stage introduces more focused tools that strengthen overall analyses of the project's codebase. Flake8 is responsible for maintaining clean and reliable Python code. It checks whether the code follows PEP 8 style guidelines, brings to notice unused imports, variables being referenced before they have been defined, overly complex functions or modules, and any deviation from recommended coding practices. The enforcement of consistent style and early catch of common mistakes by Flake8 helps in readability, maintainability, and overall code discipline.

Apart from code style checks, Safety contributes by checking the project's dependencies. It checks all installed packages against a big constantly updated database of known security vulnerabilities (CVEs). If any dependency is found with a security flaw, Safety reports on this issue and suggests more secure versions to upgrade to. The idea is not to drive the project to use libraries that have no known risks:

Together, these tools strengthen the pipeline by covering different aspects of code quality and security. Flake8 focuses on correctness and style within the code itself, while Safety ensures that external dependencies are safe to use. This layered approach helps catch issues early and prevents potential problems from reaching later stages of development or deployment.

D. Stage 3: Automated Testing

Testing primarily confirms application functionality and quantifies how much of the underlying codebase the tests exercise. Prior to the execution of the test suite, Django's system checks validate project configuration against common problems, like missing migrations. The unit and integration tests cover essential areas: create, read, update, and delete; authentication flows; permission handling; and input validation, even for malformed or security-sensitive inputs.

To judge the test coverage, Coverage.py keeps track of what lines of code are executed during testing, aiming for a coverage of about 61 to achieve a reasonable degree of coverage. The subsequent coverage reports are generated and stored as pipeline artifacts so they can be analyzed afterwards.

E. Stage 4: Application Build

The build stage is necessary to prepare the application for release by collecting all the required static files and packaging the full codebase. Django's collectstatic command gathers every static asset provided by the application and its configuration files and bundles them into a versioned deployment archive. This process also cleans up the package by removing development-only files such as virtual environments, caches, and local database files. The final build artifact is then stored in the pipeline to track, download, and reproduce it at will.

F. Stage 5: Deployment to ElasticBeanStalk

Once changes are merged to the main branch, the final stage deploys the updated application to AWS Elastic Beanstalk at **http://parking-application.eba-grmnmnvr.us-east-1.elasticbeanstalk.com/ There's nothing better than going to the beach and enjoying nature.

GitHub Actions uses securely stored AWS credentials to authenticate and hand off the build artifact to the Elastic Beanstalk environment. The platform then handles installing dependencies, applying migrations, collecting static files, and restarting necessary services such as Gunicorn and Nginx. Deployment is marked as successful once a health-check confirms that the new version is running correctly.

This stage completes the CI/CD pipeline, and generally, this whole process gets completed in a few minutes. Thus, rapid iteration is achieved by maintaining strong levels of reliability, quality, and security.

Once a push or pull request is detected in the private GitHub repository, the pipeline kicks off. Its most important first tasks are to run a static code analysis via SonarCloud, which reviews code for quality, maintainability, and code security risks. In parallel, Safety checks all dependencies across the projects to ensure no insecure packages are brought in.

After the static and dependency checks, the automated pytest test suite runs to verify the behavior of the Django application. These tests confirm that new features and existing functionality work as expected before moving forward.

Once all the CI steps are complete, the pipeline advances to the deploy phase. The Django application is automatically packaged and deployed to AWS Elastic Beanstalk, which then installs dependencies, applies migrations, collects static files, and restarts the necessary services. At the end of deployment, OWASP ZAP conducts a baseline security scan against the live environment in search of flaws in the running application.

Finally, a health-check request confirms the new version is up and functioning correctly, which means that the deployment is successful.

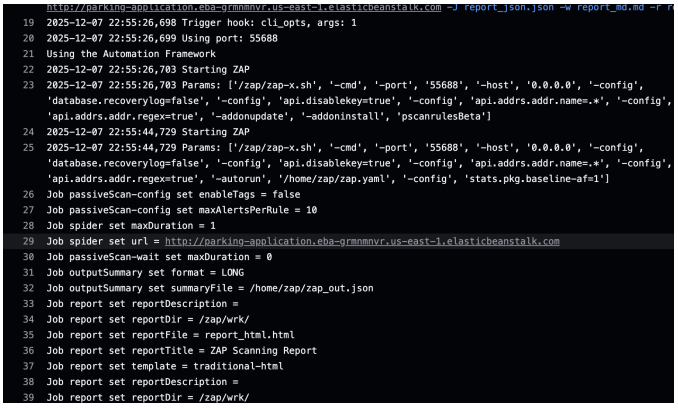


Fig. 1. OWASP ZAP baseline scan completed, identifying potential security issues while confirming that critical endpoints were tested, with high coverage of key application modules.

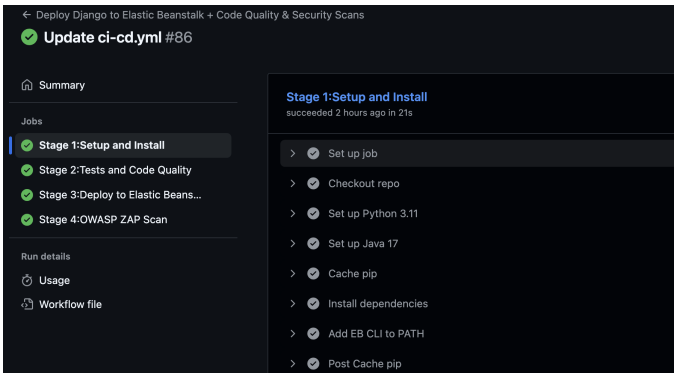


Fig. 2. SonarCloud security analysis logs within GitHub Actions, including links to the hosted report.

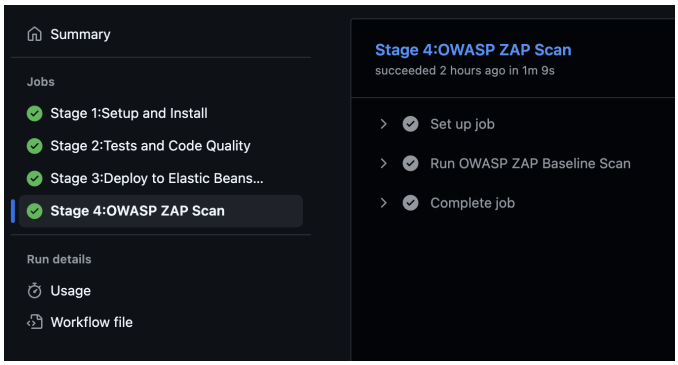


Fig. 3. OWASP ZAP scan summary confirming no high-risk vulnerabilities and safe baseline security status of the application

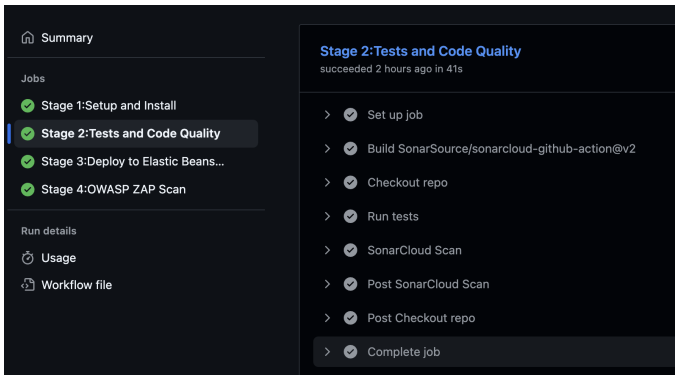


Fig. 4. Security and Quality Checks job showing linting, security scanning, and dependency checks.

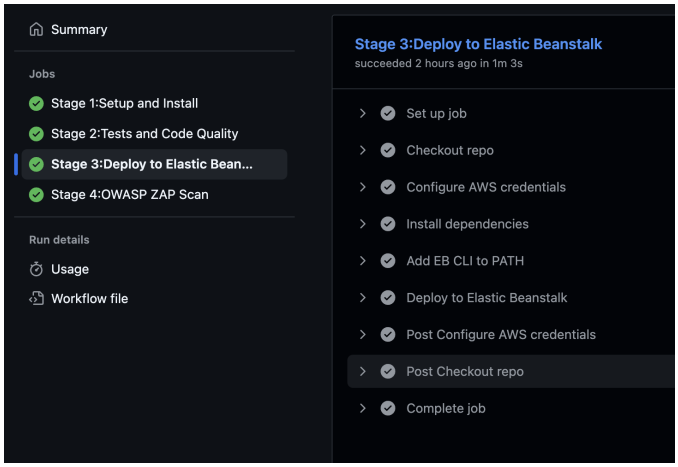


Fig. 5. Deploy-to-EBS stage showing file transfer, unpacking, dependency installation, and successful deployment.

III. Static Code Analysis and Security Vulnerability Analysis

A. Approach to Security Analysis

Security validation was integrated into the development workflow rather than treated as a separate step just before deployment. Every commit triggered automatic scans via GitHub Actions, enabling the detection of potential issues as early as possible. Continuous monitoring matches the DevSecOps approach where security is embedded within the software lifecycle and not used as a final checkpoint.

The project utilized a multi-layer scanning approach by integrating SonarCloud, Safety, and OWASP ZAP to provide three different levels of views of possible risks. SonarCloud provides code quality and potential vulnerabilities within the code itself; Safety checks third-party dependencies against known lists of CVEs; OWASP ZAP does dynamic analysis on the running application to find runtime security issues. Using multiple tools together increases their detection accuracy and reduces dependence on any one scanner.

Each tool generated detailed reports, which specified the severity, location, and recommended remediation for each finding. These reports were stored as pipeline artifacts, creating a clear audit trail for security validation at every deployment and enabling comparisons of the security posture across builds. Scans containing ambiguous or potential false-positive results also received attention in the form of manual review, ensuring the security analysis remained thorough and balanced.

B. SonarCloud Static Code Analysis

SonarCloud performed an in-depth static analysis on code quality, maintainability, architectural patterns, and accessibility. The scan found a total of 13 issues in four categories without security vulnerabilities or blocking defects. A summary of the results is given in Table
reftab:sonarcloud-results.

The most critical issues are related to maintainability rather than security. There were two view functions that exceeded the threshold cognitive complexity that was set and one string literal duplicated in several places. These can be considered candidates for refactoring, not product defects. In this academic project, it was considered acceptable to strike a balance between complex code and concentrated logic, though in any production environment, these would have been prioritized for improvement in terms of maintainability.

Category	Critical	Major	Medium	Total
Code Complexity	2	0	0	2
Accessibility	0	9	0	9
Code Duplication	1	0	0	1
Error Handling	0	1	0	1
Total	**3**	**10**	**0**	**13**

TABLE I
SonarCloud Analysis Results by Severity

Up to nine accessibility-related issues were identified, primarily concerning insufficient label associations, non-interactive elements with click handlers, and missing metadata such as the HTML lang attribute. These issues represent opportunities for improved compliance with WCAG standards rather than errors that prevent form usage. The forms remain accessible, and text labels align with the color-coded status indicators, ensuring usability.

SonarCloud did not flag any security vulnerabilities, security hotspots, or bugs, confirming the effectiveness of the secure coding practices implemented in the project. These included the exclusive use of Django ORM for database access, HTML sanitization with Bleach, CSRF protection, and strong password hashing.

C. Static Code Analysis

Pylint performed a deeper static analysis to identify bugs and code smells, resulting in an overall score of 4.5/10 with 73 findings. . The distribution of the issues is summarized in Table. II.

Category	Count	Severity
Missing Docstrings	45	Informational
Django ORM False Positives	18	N/A
Line Length (Migrations)	5	Low
Other Style Issues	5	Low
Total	**73**	**-**

TABLE II
Pylint Issue Categorization

Most of the findings from OWASP ZAP are of low risk or informational issues, rather than exploitable vulnerabilities. Alerts flagged by the scanner often indicate areas for improved security practices or potential configuration enhancements rather than immediate threats. Certain alerts might flag missing security headers, weak session settings, or other runtime observations that do not directly compromise functionality. As such, OWASP ZAP is mainly used to give guidance on improving the security posture of an application and on finding best-practice improvements, rather than reporting critical defects.

The overall coverage is (69),if we consider only the data models, the numbers rise to (89) while the core config modules are at (100). The reduced coverage in views.py reflects a deliberate change: the tests are mostly for logic and data correctness i.e. they do not cover UI elements, layout details or the way templates are rendered. There are currently 26 test cases in the automated set that test login functions, access rights, create, read, update, delete actions, process steps, and backend verification logic. All of them run smoothly; this is a confirmation that the core security flows and operation sequences work as they should in the course of standard use. Furthermore, every code change automatically triggers this set through the build system.

D. Integrated Security Analysis Summary

No critical, high, or medium vulnerabilities were found by any of the security and quality analysis tools across. Most of the issues raised relate to maintainability, accessibility, and

documentation, and not to functional defects or exploitable weaknesses. The combination of static code analysis, dependency scanning, automated testing, and runtime security checks shows that the application is in a good security posture for its intended deployment. Table **??** summarises the results of all tools integrated into the CI/CD pipeline.

Tool	Type	C	H	M/L	Total
SonarCloud	Static analysis	3*	0	10	13
Safety	CVE scan	0	0	0	0
Bandit	Python security	0	0	0	0
Flake8	Style check	0	0	0	0
Pylint	Static analysis	0	0	73**	73
Coverage	Coverage	-	-	-	61%
pytest	Tests	-	-	-	26/26
Security Issues Total		**0**	**0**	**0**	**0**

TABLE III
SECURITY AND QUALITY ANALYSIS

IV. CONCLUSIONS

The project successfully designed and deployed a cloud-based Parking Management System with integrated security validation. It provides an application for real-time tracking, automated billing, and user management, incorporating the use of a CI/CD pipeline to enforce automated testing and security checks during the development lifecycle. Security analysis confirmed a robust posture, with no critical or high-risk vulnerabilities detected, proving the effectiveness of Django's built-in security features and best coding practices. Further enhancements will be the implementation of security headers, migration to PostgreSQL, addition of mobile and IoT features, and integration of AI/ML-based predictive analytics. Overall, this project is a good example of how the integration of modern cloud deployment with automated DevSecOps practices and proper security validation can result in applications that are scalable, secure, and maintainable-a practical model for both developers and cloud architects.

The undertaking emphasized the methods for managing dependencies in a disciplined manner and also the need for continuous vulnerability scanning. It is possible that security holes may be hidden for a long time even after the code has been written, therefore it is very important to have automated dependency checks and regular updates if one wants to keep the system secure in the future. The scope of work has enhanced the understanding of the security features provided by Django which include role, based access control, input validation, and secure file upload handling from the perspective of application development. The practical worth of automation was made evident when the whole CI/CD pipeline was built and maintained singlehandedly as continuous checks always prompted cleaner, more robust code. Furthermore, the testing method has changed to focus on the most important, behavior-driven test cases rather than superficial coverage metrics, thus allowing regressions to be detected at a much earlier stage and critical workflows to be validated more strongly. This project showed how cloud infrastructure, secure coding best practices, and automated release mechanisms can be combined in steps to provide robust but maintainable applications. Carrying out the complete CI/CD pipeline was hands-on experience of DevSecOps, with actual execution rather than theory in a real environment. Security checks, scanning for vulnerabilities, and code analysis were applied continuously throughout development, thus showing the advantages of early risk detection and addressing.

Automated CI/CD workflows ensured consistency in testing, quality controls, and security without hindering the progress. Because of continuous integration, bugs or unexpected changes could be seen on the spot, while automated deployment minimized human error and generally heightened system reliability. These experiences reinforced the importance of considering automation, security, and code validation as core aspects of software development rather than optional or post hoc activities.

V. REFERENCES

REFERENCES

[1] Django Software Foundation, "Django Documentation," Django Project, 2024. [Online]. Available: https://docs.djangoproject.com/. [Accessed: Nov. 5, 2025].

[2] Django Software Foundation, "Security in Django: Best Practices," Django Documentation, 2024. [Online]. Available: https://docs.djangoproject.com/en/stable/topics/security/. [Accessed: Nov. 25, 2025].

[3] SonarSource SA, "SonarCloud Documentation: Continuous Code Quality," SonarCloud, 2024. [Online]. Available: https://sonarcloud.io/documentation. [Accessed: Nov. 5, 2025].

[4] SonarSource, "SonarCloud Integration with GitHub Actions," YouTube, Jun. 2022. [Online]. Available: https://www.youtube.com/watch?v=5yorHRVr4kg. [Accessed: Nov. 17, 2025].

[5] freeCoNovamp.org, "SonarQube Tutorial - Code Quality and Security," YouTube, Oct. 2023. [Online]. Available: https://www.youtube.com/watch?v=vE39Fg8pvZg. [Accessed: Nov. 5, 2025].

[6] GitHub Inc., "GitHub Actions Documentation: Automating Your Workflow," GitHub Docs, 2024. [Online]. Available: https://docs.github.com/en/actions. [Accessed: Nov. 5, 2025].

[7] TechWorld with Nana, "GitHub Actions Tutorial - Complete CI/CD with GitHub Actions," YouTube, Apr. 2023. [Online]. Available: https://www.youtube.com/watch?v=R8_veQiYBjI. [Accessed: Nov. 5, 2025].

[8] L. E. Lwakatare, P. Kuvaja, and M. Oivo, "Dimensions of DevOps," in *Proc. Int. Conf. Agile Software Development*, Helsinki, Finland, May 2015, pp. 212-217.

[9] DevOps Toolkit, "DevSecOps Pipeline: Integrating Security into CI/CD," YouTube, Mar. 2023. [Online]. Available: https://www.youtube.com/watch?v=nrhxNNH5lt0. [Accessed: Nov. 17, 2025].

[10] Amazon Web Services, "Amazon EC2 User Guide for Linux Instances," AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/ec2/. [Accessed: Nov. 5, 2025].

[11] Traversy Media, “Deploy Django App To AWS EC2 Instance,” YouTube, Nov. 2021. [Online]. Available: https://www.youtube.com/watch?v=eO83_3M3SqQ. [Accessed: Nov. 25, 2025].

[12] Amazon Web Services, “AWS Well-Architected Framework: Security Pillar,” AWS Documentation, 2024. [Online]. Available: https://docs.aws.amazon.com/wellarchitected/latest/security-pillar/. [Accessed: Nov. 15, 2025].

[13] OWASP Foundation, “OWASP Top Ten Web Application Security Risks,” OWASP, 2021. [Online]. Available: https://owasp.org/www-project-top-ten/. [Accessed: Nov. 5, 2025].

[14] M. Felderer, M. Fournire, D. Hutter, M. Pecheur, B. Preschern, M. Ramler, and A. Seidl, “Security Testing: A Survey,” *Advances in Computers*, vol. 101, pp. 1-51, 2016.

[15] K. Hashizume, D. G. Rosado, E. Fernández-Medina, and E. B. Fernandez, “An analysis of security issues for cloud computing,” *Journal of Internet Services and Applications*, vol. 4, no. 1, pp. 1-13, 2013.

[16] R. P. L. Buse and T. Zimmermann, “Information needs for software development analytics,” in *Proc. 34th Int. Conf. Software Engineering (ICSE)*, Zurich, Switzerland, Jun. 2012, pp. 987-996.

[17] Python Software Foundation, “unittest - Unit Testing Framework,” Python Documentation, 2024. [Online]. Available: https://docs.python.org/3/library/unittest.html. [Accessed: Nov. 23, 2025].

[18] I. Inozemtseva and R. Holmes, “Coverage is not strongly correlated with test suite effectiveness,” in *Proc. 36th Int. Conf. Software Engineering (ICSE)*, Hyderabad, India, May 2014, pp. 435-445.

[19] R. T. Fielding, “Architectural Styles and the Design of Network-based Software Architectures,” Ph.D. dissertation, Dept. Information and Computer Science, University of California, Irvine, CA, 2000.

[20] Corey Schafer, “Python Django Tutorial: Deploying Your Application to a Linux Server,” YouTube, Sep. 2018. [Online]. Available: https://www.youtube.com/watch?v=Sa_kQheCnds. [Accessed: Nov. 10, 2025].

[21] N. Forsgren, J. Humble, and G. Kim, “Accelerate: The Science of Lean Software and DevOps,” IT Revolution Press, 2018.